[PDyTR][2026] Expl. 4

JADE

Cuando lo que viaja es el programa

Programación Distribuida y Tiempo Real · Explicación TP4

Gastón Marón · Facultad de Informática UNLP

Para arrancar

¿Qué viaja por la red?

Modalidad	Qué viaja por la red	Quién espera a quién
Cliente/servidor		
MQTT		
JADE	**?**	**?**

Ya vimos dos formas de comunicar procesos. Completemos lo que sabemos de cada una.

Para arrancar

¿Qué viaja por la red?

Modalidad	Qué viaja por la red	Quién espera a quién
Cliente/servidor	Pedido y respuesta (datos)	El cliente espera al servidor
MQTT		
JADE	**?**	**?**

Ya vimos dos formas de comunicar procesos. Completemos lo que sabemos de cada una.

Para arrancar

¿Qué viaja por la red?

Modalidad	Qué viaja por la red	Quién espera a quién
Cliente/servidor	Pedido y respuesta (datos)	El cliente espera al servidor
MQTT	Mensajes, a través de un broker	El suscriptor espera; el publicador, no
JADE	**?**	**?**

Ya vimos dos formas de comunicar procesos. Completemos lo que sabemos de cada una.

Idea central

En JADE se mueve el proceso

Modalidad	Qué viaja por la red	Quién espera a quién
Cliente/servidor	Pedido y respuesta (datos)	El cliente espera al servidor
MQTT	Mensajes, a través de un broker	El suscriptor espera; el publicador, no
JADE	El programa, con sus datos	El agente va, trabaja y vuelve

En cliente/servidor y MQTT se mueven mensajes entre procesos quietos. En JADE viaja el programa.

01

Mover un programa

Mover un programa

¿Para qué mover código?

1

Eficiencia

Repartir la carga de cómputo y mandar menos datos por la red: se procesa donde están los datos.

2

Código a demanda

Ejecutar código que el destino no conocía de antemano, sin instalarlo en cada máquina.

3

Respuesta

Acercar el cómputo al usuario para responder más rápido.

En el TP: recolectar el estado de muchas máquinas sin instalar un servidor en cada una.

Mover un programa

Tres capas que pueden viajar

Código
Las clases
el .class que define qué sabe hacer el programa

Datos
Los atributos
los valores que el objeto tiene guardados

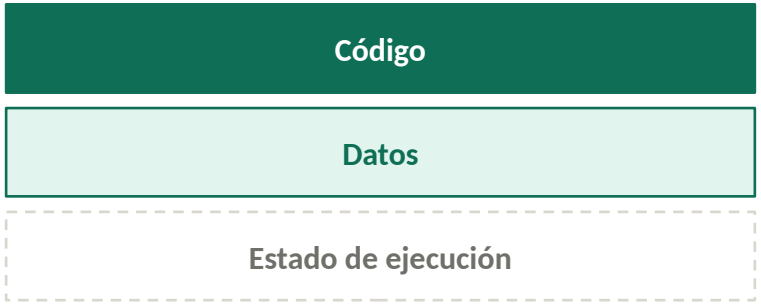
Estado de ejecución
Dónde va la ejecución
la línea actual, la pila y las variables locales

La pregunta de la clase, más precisa: ¿cuáles de estas capas llegan al destino?

Mover un programa

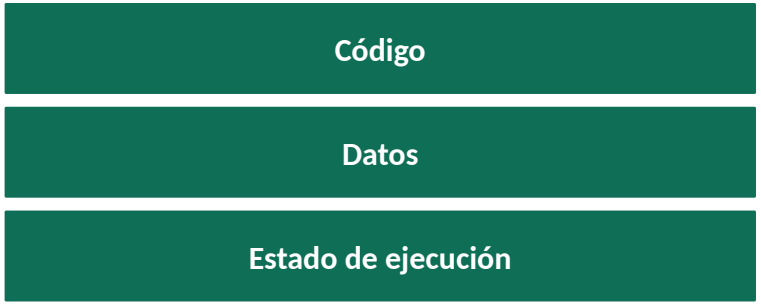
Movilidad débil y fuerte

Movilidad débil

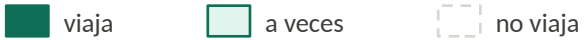


El destino arranca desde un punto de entrada

Movilidad fuerte



El programa sigue en la línea donde estaba



La diferencia decide si el destino arranca de nuevo o continúa donde estaba.

Mover un programa

Quién inicia y qué queda en el origen

¿Quién inicia?

Proactiva
El origen manda el código

Reactiva
El destino lo pide

¿Qué queda en el origen?

Migrar
Nada: el proceso deja de existir allá

Clonar
Una copia: ambos siguen activos

En el TP, AgenteA decide la migración de AgenteB y AgenteB no deja copia: migra.

Mover un programa

Los cuatro casos

	Inicia el origen	Inicia el destino
Movilidad débil	Se envía código para que el destino lo ejecute	El destino pide y descarga el código que necesita
Movilidad fuerte	Un proceso en ejecución se muda o se clona	El destino reclama un proceso en ejecución

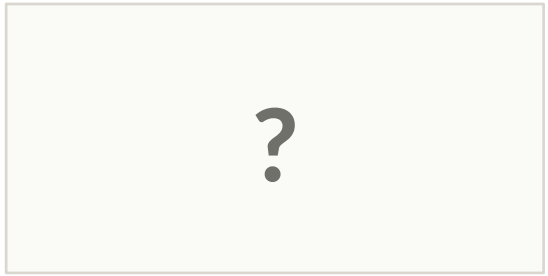
Clasificación de Tanenbaum. ¿En qué casillero cae JADE? Votamos ahora y de nuevo después de la demo.

02

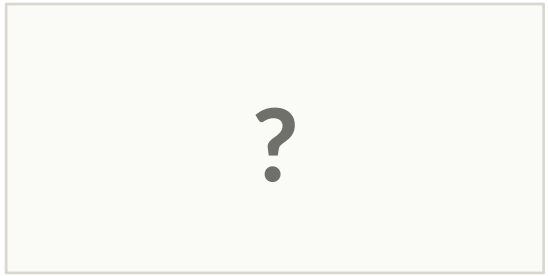
JADE

JADE

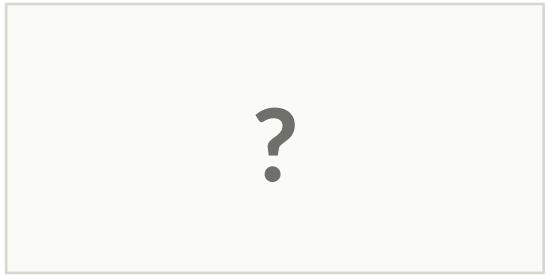
Ojo con los nombres



Plataforma



Contenedor

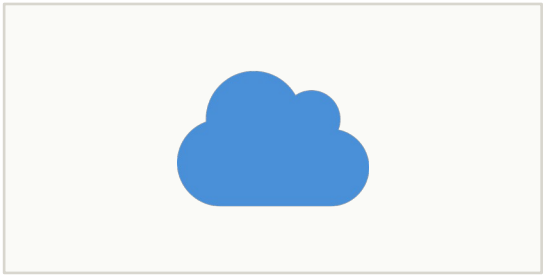


Agente

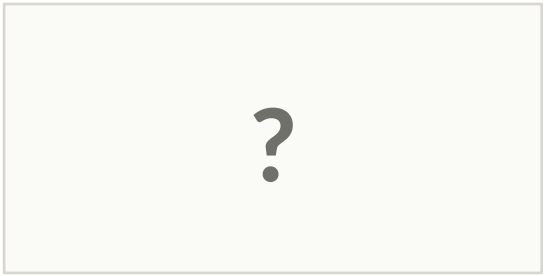
¿Qué se te viene a la cabeza con cada palabra?

JADE

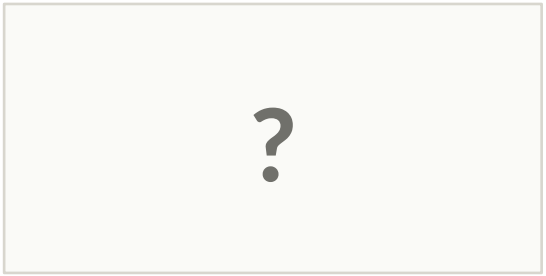
Ojo con los nombres



Plataforma



Contenedor



Agente

¿Qué se te viene a la cabeza con cada palabra?

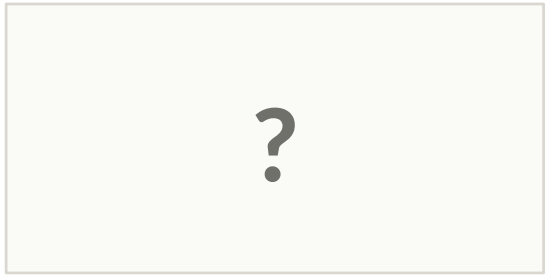
JADE

Ojo con los nombres

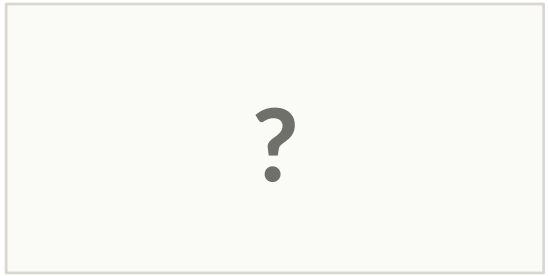


Plataforma

En JADE: el conjunto de contenedores que forman un sistema



Contenedor



Agente

¿Qué se te viene a la cabeza con cada palabra?

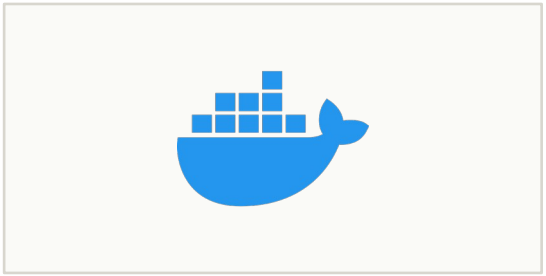
JADE

Ojo con los nombres

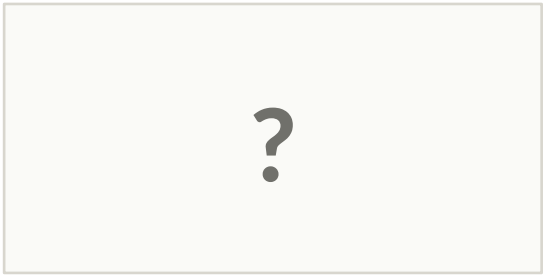


Plataforma

En JADE: el conjunto de contenedores que forman un sistema



Contenedor



Agente

¿Qué se te viene a la cabeza con cada palabra?

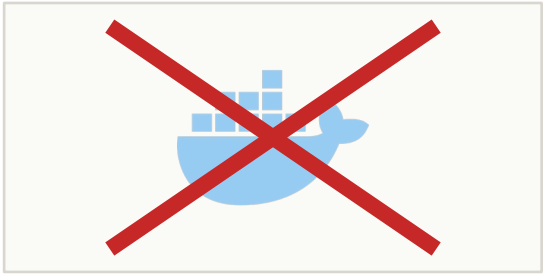
JADE

Ojo con los nombres



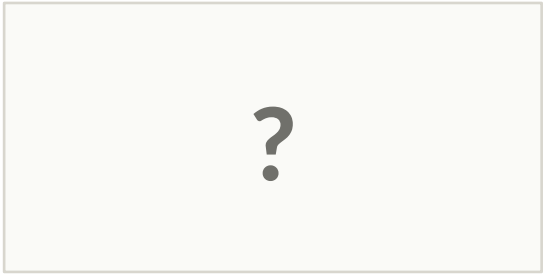
Plataforma

En JADE: el conjunto de contenedores que forman un sistema



Contenedor

En JADE: el ambiente de ejecución (una JVM) donde viven los agentes



Agente

¿Qué se te viene a la cabeza con cada palabra?

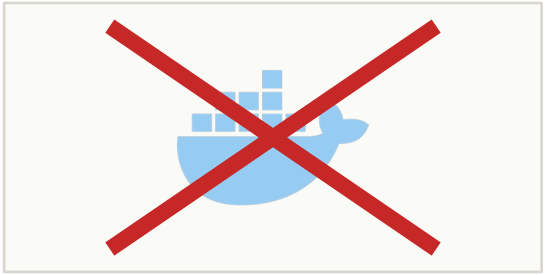
JADE

Ojo con los nombres



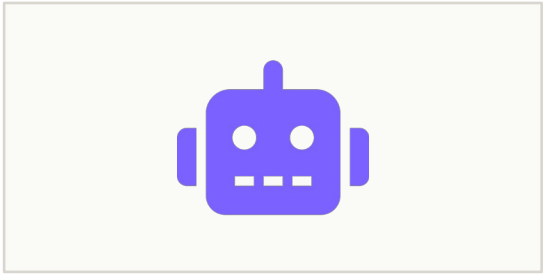
Plataforma

En JADE: el conjunto de contenedores que forman un sistema



Contenedor

En JADE: el ambiente de ejecución (una JVM) donde viven los agentes



Agente

¿Qué se te viene a la cabeza con cada palabra?

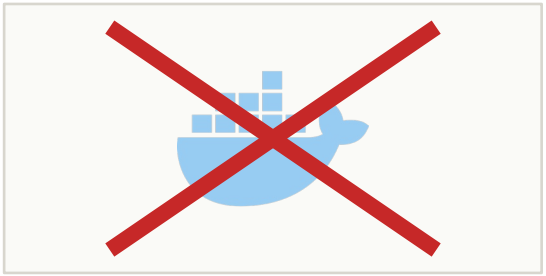
JADE

Ojo con los nombres



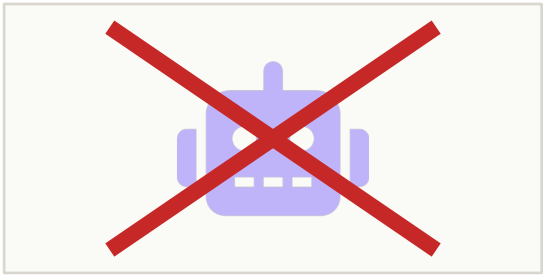
Plataforma

En JADE: el conjunto de contenedores que forman un sistema



Contenedor

En JADE: el ambiente de ejecución (una JVM) donde viven los agentes



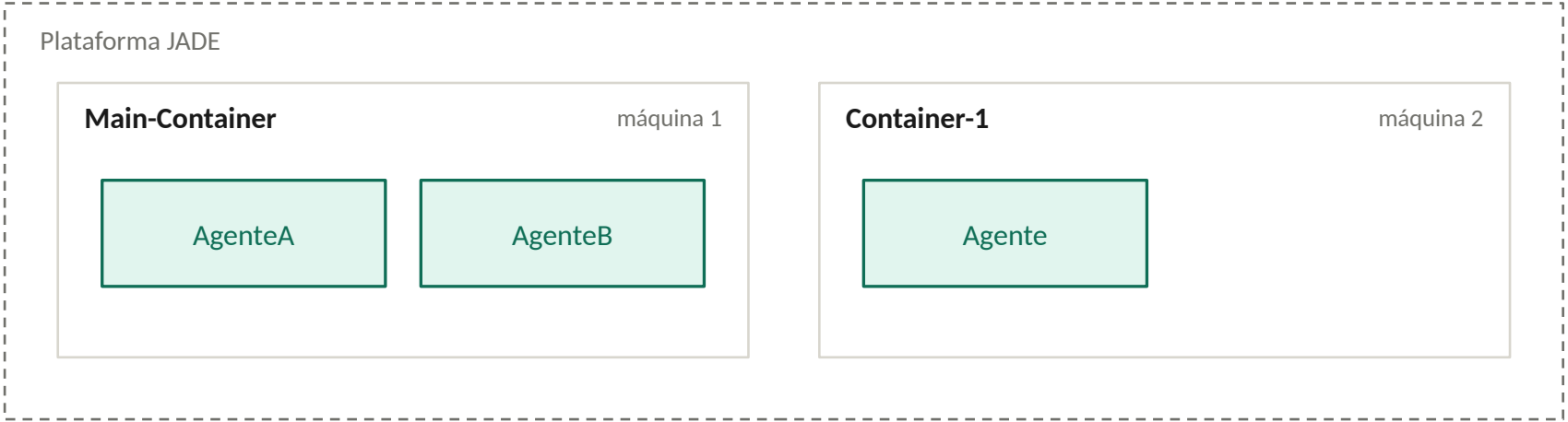
Agente

En JADE: un objeto Java autónomo que se puede mudar

JADE es del año 2000: los nombres llegaron antes que la nube, Docker y la IA generativa.

JADE

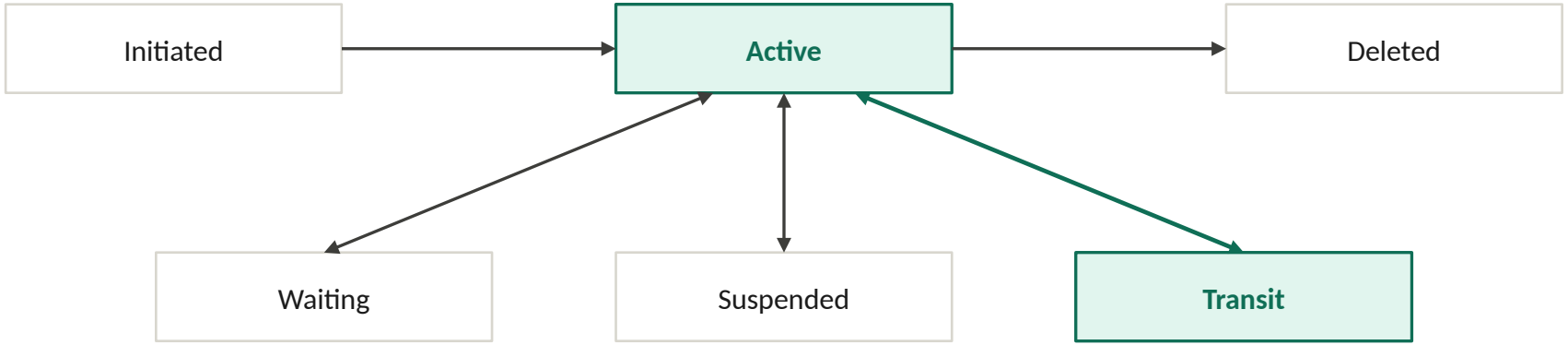
Plataforma, contenedores y agentes



Contenedor: el ambiente de ejecución del agente. Plataforma: el conjunto de contenedores, que suelen estar en máquinas distintas.

JADE

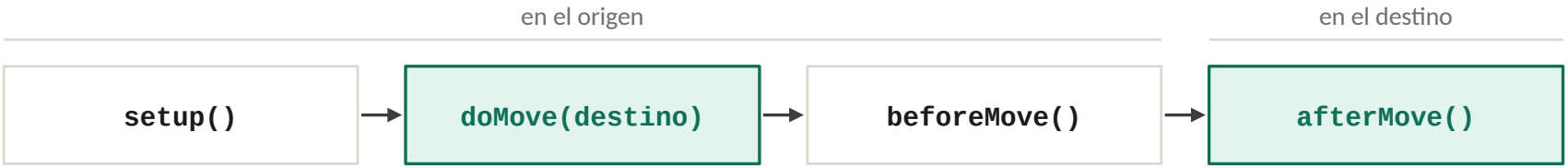
Ciclo de vida de un agente



Transit es el estado por el que pasa el agente mientras migra de un contenedor a otro.

JADE

Los métodos que importan



lo llama JADE
al crear el agente. Al migrar, no se vuelve a llamar

lo llama el agente
pide la migración: se concreta al terminar el método en curso

lo llama JADE
en el origen, justo antes de partir

lo llama JADE
en el destino, apenas llega

doMove() no mueve al agente en ese instante: deja anotado el pedido y JADE lo hace al terminar el método.

03

Ver la movilidad

Ver la movilidad

Antes de correrlo: ¿qué va a pasar?

```
public class DemoMovilidad extends Agent {
  private int contador = 0;        // atributo
  private transient String nota;   // transient

  protected void setup() {
    int local = 42;                // variable local
    contador++;
    nota = "anotado en el origen";
    doMove(new ContainerID("Main-Container", null));
    log("línea siguiente a doMove()");
  }

  protected void afterMove() {
    contador++;
    log("afterMove() contador=" + contador + " nota=" + nota);
  }
}
```

1. **¿Corre la línea después de doMove()?** ¿En qué contenedor?
2. **¿Cuánto vale contador en afterMove()?**
3. **¿Cuánto vale nota al llegar?**
4. **¿Se vuelve a ejecutar setup() al llegar?**

Un minuto solos, dos en pareja, y votamos. El agente arranca en Container-1.

Ver la movilidad

La demo

Compilar

```
mkdir -p classes && javac -classpath lib/jade.jar -d classes myexamples/DemoMovilidad.java
```

Terminal 1 · Main-Container (sin classes en el classpath)

```
java -cp lib/jade.jar jade.Boot -gui -local-host 127.0.0.1
```

Terminal 2 · Container-1 con el agente

```
java -cp lib/jade.jar:classes jade.Boot -container -local-host 127.0.0.1 \
     -host 127.0.0.1 -agents demo:DemoMovilidad
```

El Main-Container no tiene la clase DemoMovilidad. Fijate qué pasa igual.

Ver la movilidad

Lo que vimos

```
[host | Container-1]    setup()  contador=1  local=42  nota=anotado en el origen
[host | Container-1]    línea siguiente a doMove()
[host | Main-Container] afterMove()  contador=2  nota=null
```

Viajó

- El código: el Main-Container no tenía la clase
- Los atributos: contador llegó en 1 y pasó a 2

No viajó

- La variable local y lo transient (nota = null)
- Dónde iba la ejecución: arranca afterMove()

La línea siguiente a doMove() corrió en el origen: el agente se fue recién al terminar setup().

Ver la movilidad

JADE: débil, con estado de datos

	+ datos	Inicia el origen	Inicia el destino
Movilidad débil	Se envía código para que el destino lo ejecute		El destino pide y descarga el código que necesita
Movilidad fuerte	Un proceso en ejecución se muda o se clona		El destino reclama un proceso en ejecución

Viajan el código y los atributos, no la ejecución. A veces se la llama movilidad «no tan débil».

Ver la movilidad

Para programar: lo que necesitás, en atributos

```
private List<Location> itinerario;  // viaja
private List<String> resultados;    // viaja

protected void afterMove() {
  resultados.add(medirAca());
  if (!itinerario.isEmpty())
    doMove(itinerario.remove(0));
}
```

1. **afterMove() decide**
   según lo que dicen los atributos

2. **No hay "línea siguiente"**
   en el destino: se arranca de nuevo

3. **Lo no serializable**
   (GUI, sockets, archivos) se reabre al llegar

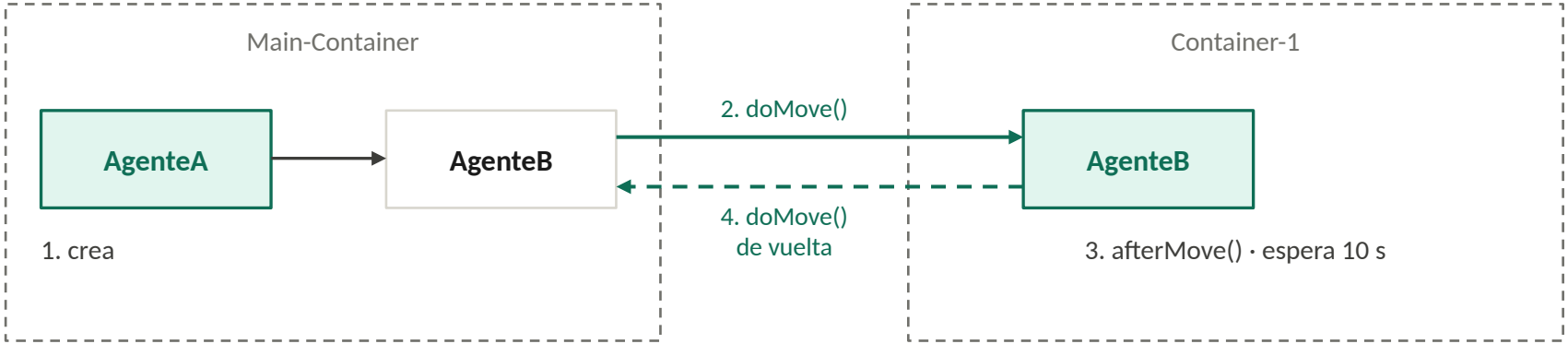
Si el agente necesita saber de dónde vino o adónde sigue, eso tiene que viajar en un atributo.

04

Del ejemplo al TP

Del ejemplo al TP

Ida y vuelta



Ej. 1 y 2: AgenteB llega con sus atributos. ¿Qué tiene que llevar para saber adónde volver?

Del ejemplo al TP

Si la migración falla, ¿quién se entera?

```
[host | Container-1]  setup()  contador=1  local=42  nota=anotado en el origen
[host | Container-1]  línea siguiente a doMove()
SEVERE: Destination Container-9 does not exist or does not support mobility
```

1. **El agente no recibe ninguna excepción**
   y sigue en el origen como si nada

2. **Si AgenteA decide la migración, ¿cómo se entera de que falló?**

Ej. 2.1 y 2b: el control de errores va en quien decide la migración, no en AgenteB.

Del ejemplo al TP

Un recorrido circular



Ej. 3: ¿cuánto tarda la vuelta? ¿El estado juntado es una foto de un mismo instante?

Del ejemplo al TP

Tres modalidades frente a los escenarios

Escenario	Cliente/servidor	MQTT	JADE
Se suma o se quita una máquina			
Además hay que recolectar la cantidad de procesos			
Tráfico de red para juntar todo			
¿Es una foto de un mismo instante?			

En grupos: cada grupo completa una fila para las tres modalidades. Es el ensayo de los ej. 4 y 5.

Para cerrar

¿Y los agentes de IA?

Un sistema de agentes de IA que se reparten una tarea: ¿a qué modalidad de la tabla se parece más?

¿Qué viaja por la red?	¿Quién espera a quién?	¿Qué estado hay que mandar a mano?

Pista: comparalo con los atributos que el agente JADE tiene que llevar consigo.

Para cerrar

Cómo es una entrega exitosa del ej. 1

- ✓ Dos computadoras distintas (una puede ser virtual), un contenedor en cada una
- ✓ La salida de cada máquina muestra el host y el contenedor en cada paso
- ✓ El informe explica qué viaja y por qué hace falta movilidad de código
- ✓ AgenteA crea a AgenteB y lo migra al otro contenedor
- ✓ AgenteB muestra que completó la migración
- ✓ Clases sin package; si usás behaviours, justificá cuál y por qué

Si el 29/10 lo traés corriendo en dos máquinas, el resto del TP se apoya en esto.

Para cerrar

Problemas frecuentes

`Node <Container-1> … is not reachable`

Usar -local-host con la IP que el otro contenedor alcanza

`Connection refused`

Firewall o IP equivocada: revisar -host y el puerto 1099

`directory not found: classes`

Crear la carpeta antes de compilar: mkdir classes

`ClassNotFoundException en Windows`

El classpath se separa con ; (ver jade.ps1)

Los comandos completos están en el Makefile y en jade.ps1 del ejemplo.

Para cerrar

Próximos pasos

Ejercicio 0 · no se entrega

1. **Correr DemoMovilidad en dos máquinas**
2. **Explicar en tres líneas la salida de cada una**
3. **Mandarlo a un contenedor inexistente**
   y anotar dónde aparece el error

Fechas

22/10	Hoy: explicación
29/10	Consultas: traer el Ejercicio 0 corriendo
05/11	Taller TP4 y explicación del parcial

¡Muchas gracias!

¿Preguntas?